



面向任意形状数据的 快速聚类算法研究

答辩人： 魏绍康

专 业： 计算机科学与技术

导 师： 黄浩 教授

时 间： 2026年5月8日



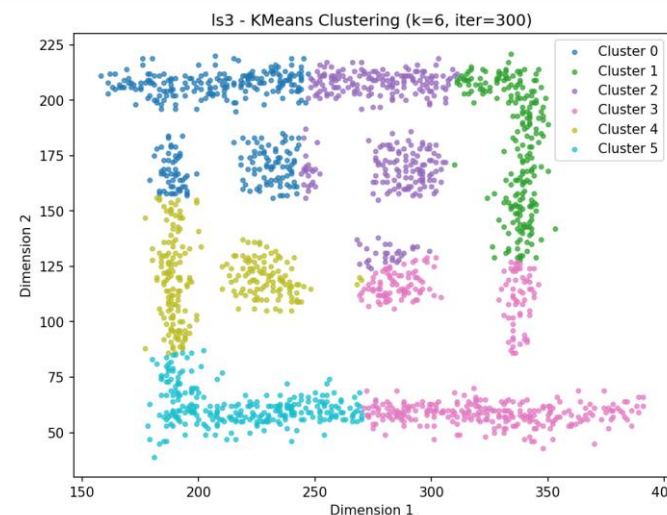
目录

1. 研究背景与意义
2. FASCA算法设计思路
3. 算法总结与分析
4. 实验环境与设置
5. 实验结果分析
6. 总结与展望

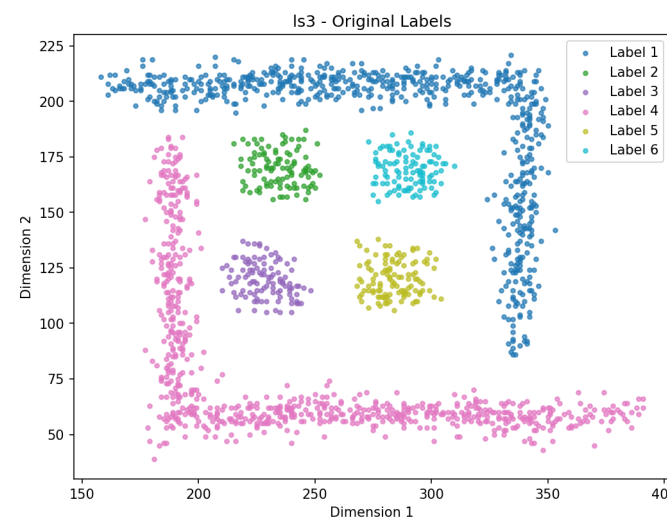
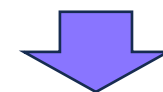


1.研究背景与意义

- **研究背景：** 现实数据往往呈现复杂的非凸、任意形状。
- **现有问题：**
 - K-Means算法只能处理球状数据。
 - 基于密度的方法处理数据效率低。
 - 随机采样方法易破坏数据拓扑结构。
- **目标：** 保证准确率、提高效率。



K-Means



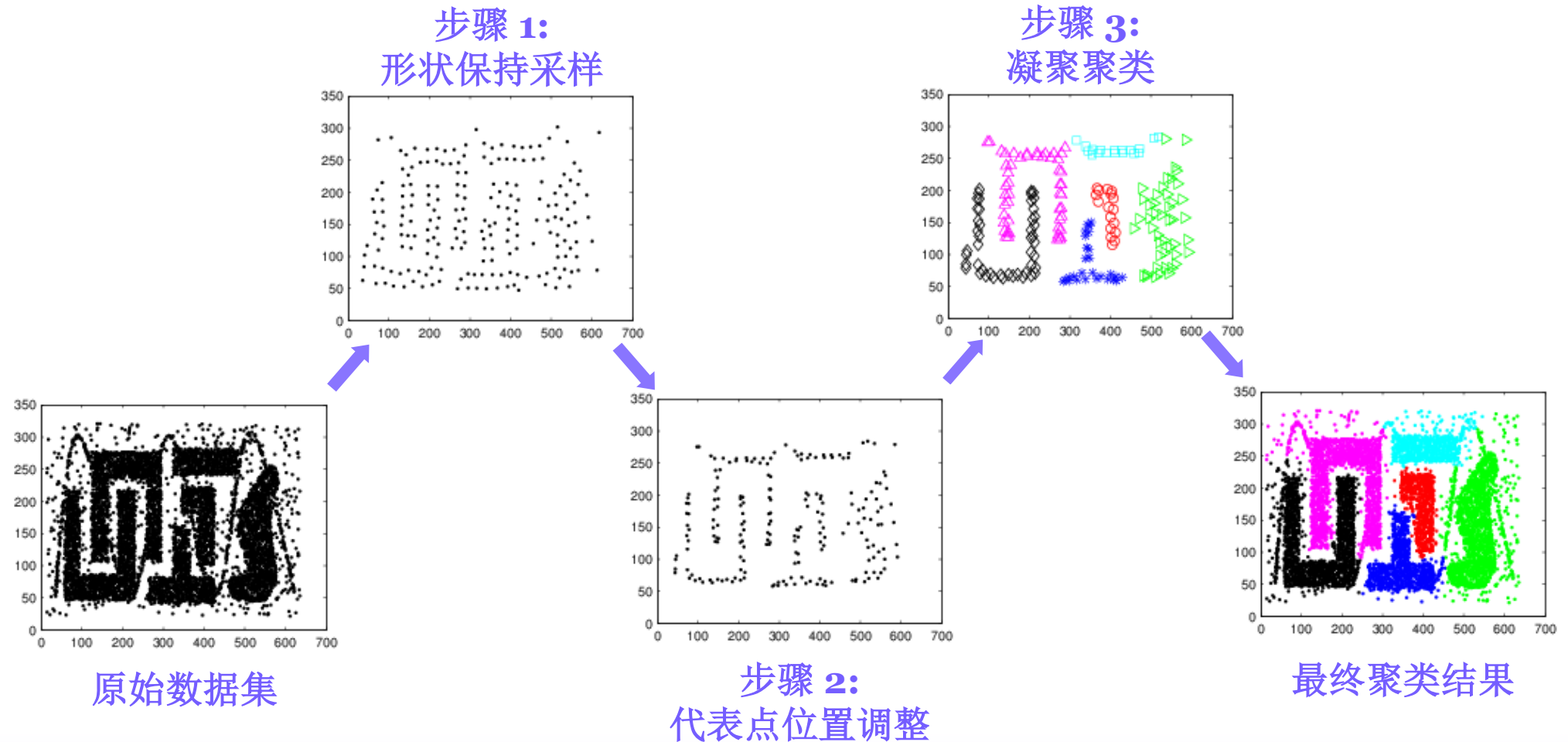
理想结果



目录

1. 研究背景与意义
2. FASCA算法设计思路
3. 算法总结与分析
4. 实验环境与设置
5. 实验结果分析
6. 总结与展望

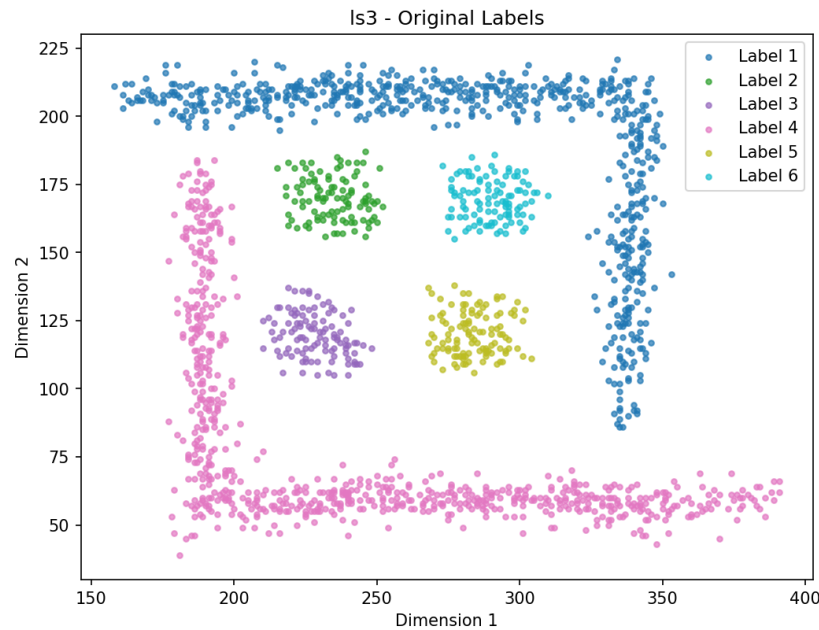
2. FASCA算法设计思路



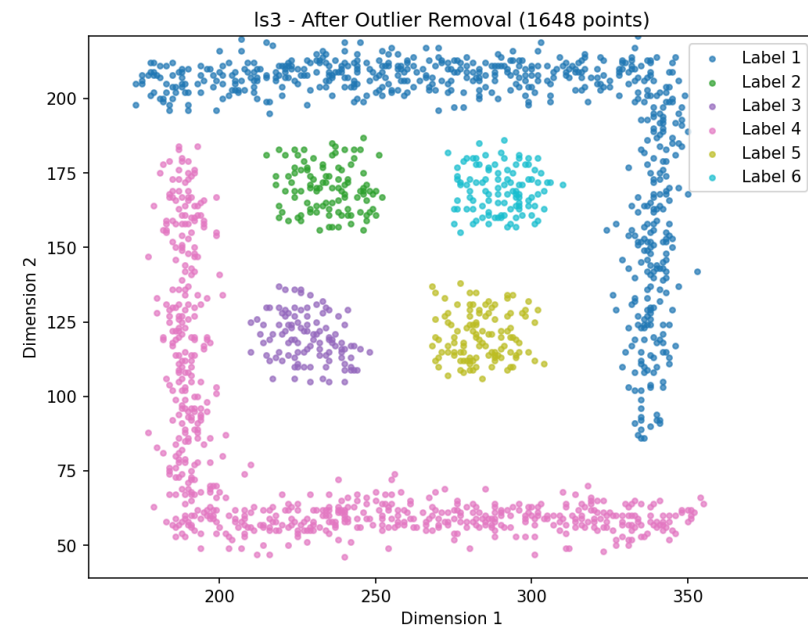
2. FASCA算法设计思路

步骤 1: 形状保持采样

- **去除离群数据:** 使用快速离群点检测算法,时间复杂度 $O(N)$,可快速去除后5%的离群数据。



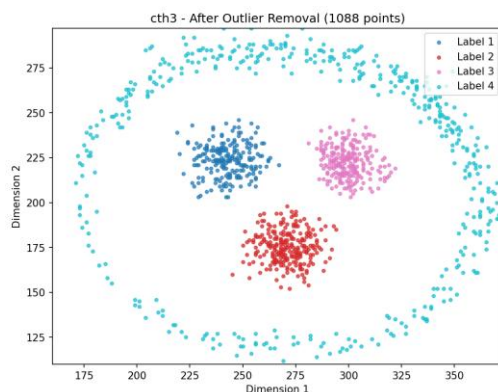
➡
去除离群点



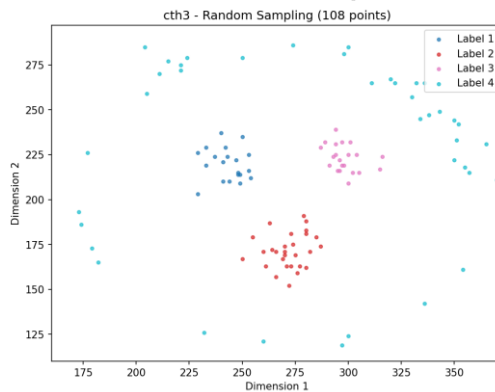
2. FASCA算法设计思路

步骤 1: 形状保持采样

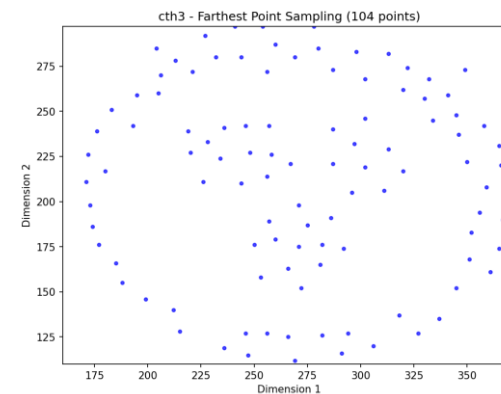
- 远端采样策略：
 - 解决痛点：随机采样容易破坏数据的拓扑结构。
 - 远端采样逻辑：
 - 选取距离中心最近的点作为第一个采样点。
 - 每次从剩余点中，选择距离已选点集合最远的那个点。
 - 当新选点的距离变化率低于阈值 ϵ 时停止。



原始数据集



随机采样



远端采样



2. FASCA算法设计思路

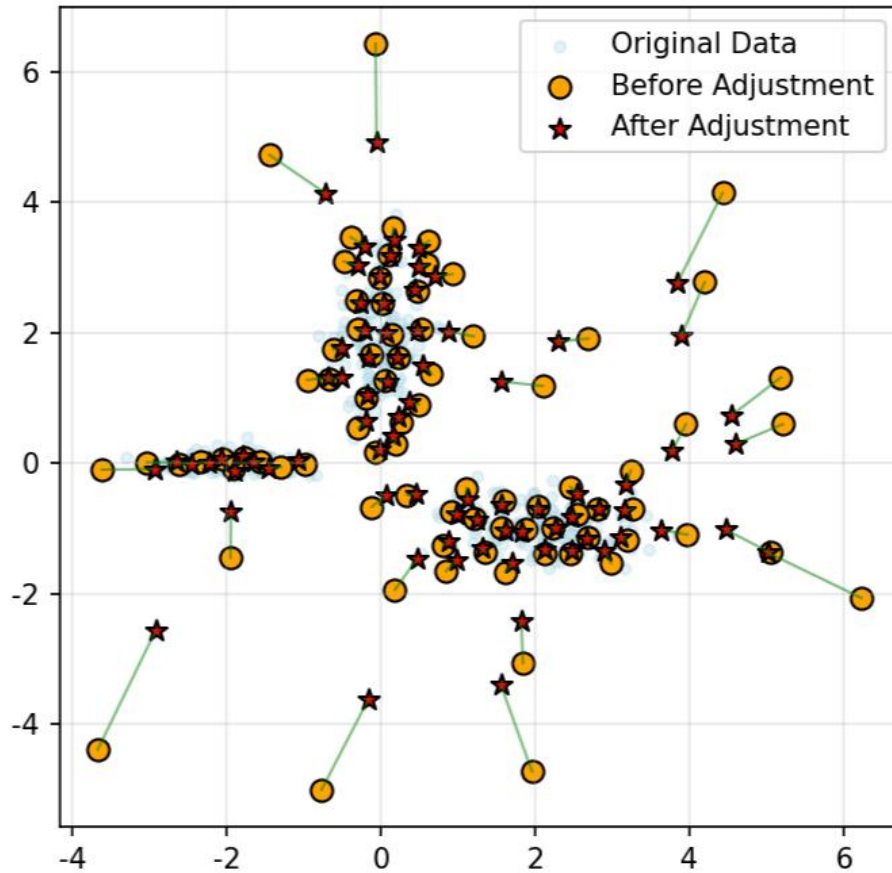
步骤 2: 代表点位置调整

- 启发：力导向模型中，样本点间互相产生作用力。
- 算法步骤：
 1. 初步聚类，为每个代表点分配一个簇标签。
 2. 为每个代表点计算k近邻集合。
 3. 在k近邻集合中借助拉普拉斯平滑思想进行混合位移计算：
 - 簇内平滑矢量：
$$V_{smooth}(i) = \frac{1}{|N_{in}(i)|} \sum_{j \in N_{in}(i)} (p_j - p_i)$$
 - 簇间排斥矢量：
$$V_{rep}(i) = \sum_{j \in N_{out}(i)} \left(\frac{d_{min} - d_{ij}}{d_{ij} + \epsilon} \cdot (p_i - p_j) \right)$$
 - 位置锚定矢量：
$$V_{anchor}(i) = p_i^{init} - p_i$$
- 多轮迭代：根据效率需求重复算法步骤。



2. FASCA算法设计思路

步骤 2: 代表点位置调整





2. FASCA算法设计思路

步骤 3: 凝聚聚类

- **聚类策略：自底向上凝聚层次聚类**
 1. 遍历当前所有簇对，计算其类间最小距离。
 2. 找到距离最小的一对簇，并将其合并为同一个簇。
 3. 重复步骤 2，直到簇的数量为预设K值。
- **标签映射：**聚类完成后，对于原始数据集中的每一个样本点，寻找调整后代表点集中距离它最近的代表点，并将该代表点所属的簇标签分配给它。

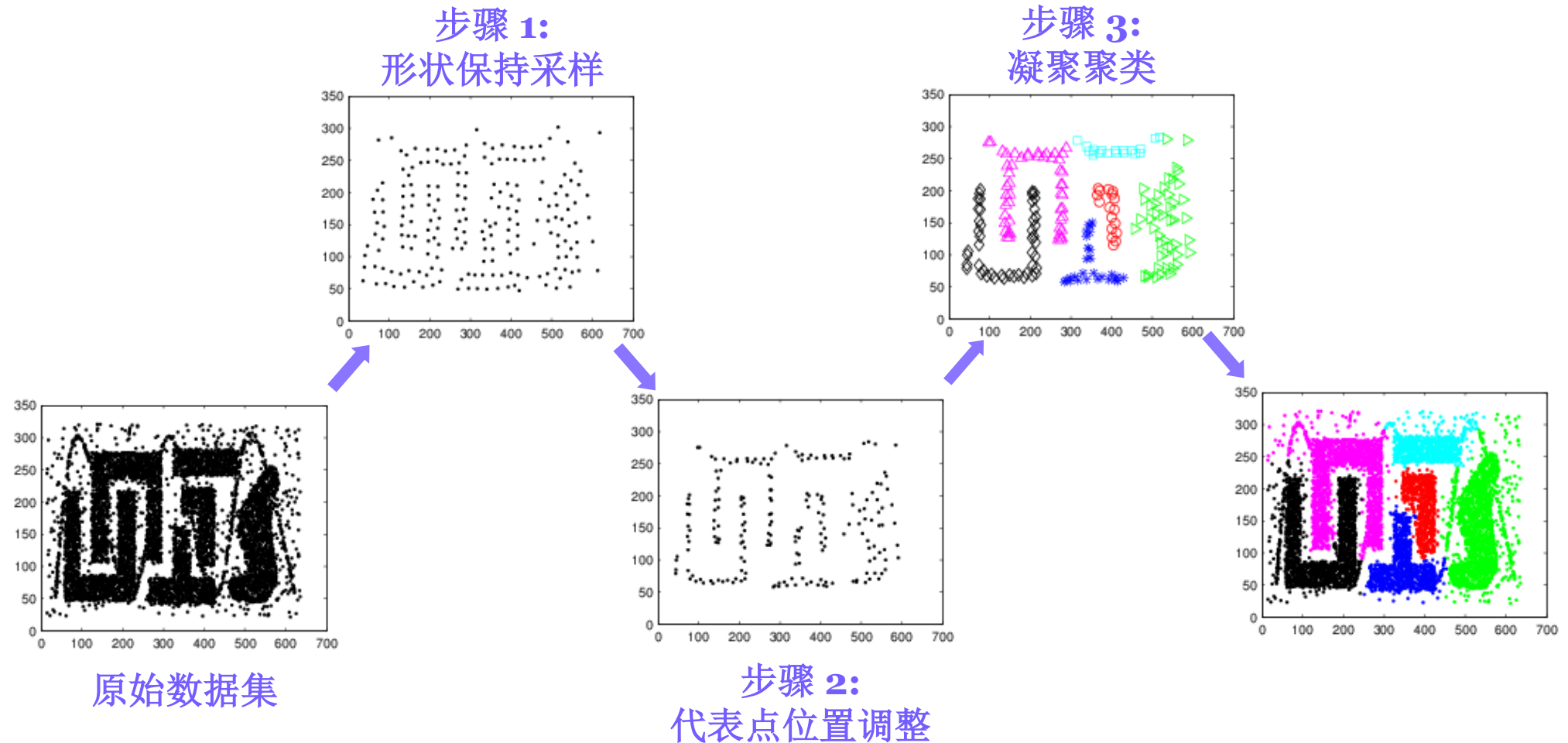


目录

1. 研究背景与意义
2. FASCA算法设计思路
3. 算法总结与分析
4. 实验环境与设置
5. 实验结果分析
6. 总结与展望



3.算法总结与分析





3.算法总结与分析

- 算法时间复杂度分析:

- 1. 形状保持采样阶段:

- 快速离群点检测算法: $O(N)$
 - 远端采样 M 个代表点: $O(N \cdot M)$ 且 $M \ll N$

- 2. 代表点位置调整阶段:

- 构建 k -NN图 $O(M \log M)$
 - 计算其 k 个邻居的受力情况: $O(T \cdot k \cdot M)$

- 3. 凝聚聚类:

- 距离矩阵优化凝聚聚类: $O(M^2)$
 - 将簇标签借助KD-tree映射到原始的 N 个点: $O(N \log M)$

- 算法整体时间复杂度: $O(N \cdot M + M^2)$



目录

1. 研究背景与意义
2. FASCA算法设计思路
3. 算法总结与分析
4. 实验环境与设置
5. 实验结果分析
6. 总结与展望



4. 实验环境与设置

- 实验环境：

处理器	AMD Ryzen 5 5500 (3.60 GHz)
内存	16GB
操作系统	Windows 11 (26200.8037)
编程语言	Python 3.10.11



4. 实验环境与设置

- 数据集：
 - 人工合成数据集：

数据集	样本个数	类个数	维度	形状分布
DS-1.1	20000	3	2	球形
DS-1.2	20000	4	2	不规则形状
DS-1.3	20000	3	2	簇内密度不均匀
DS-1.4	13000	4	2	簇间密度不一致
DS-1.5	20000	2	2	同心环



4. 实验环境与设置

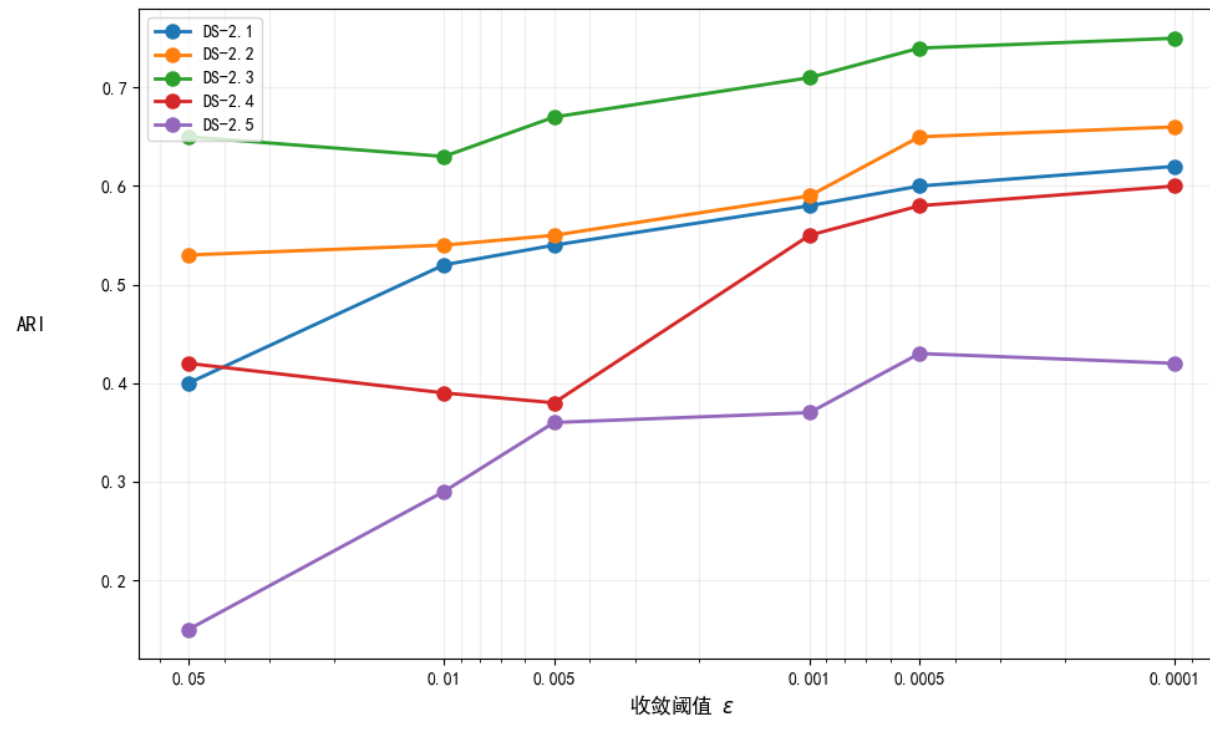
- 数据集：
 - 基准数据集：

数据集	数据集名称	样本个数	维度	来源
DS-2.1	Iris	150	4	UCI
DS-2.2	Bank Marketing	45210	17	UCI
DS-2.3	Adult	48840	14	UCI
DS-2.4	Student Performance	649	33	UCI
DS-2.5	Online Retail	541910	6	UCI
DS-2.6	Dry Bean	13610	16	UCI
DS-2.7	Credit Approval	690	15	UCI
DS-2.8	Spambase	4600	57	UCI



4. 实验环境与设置

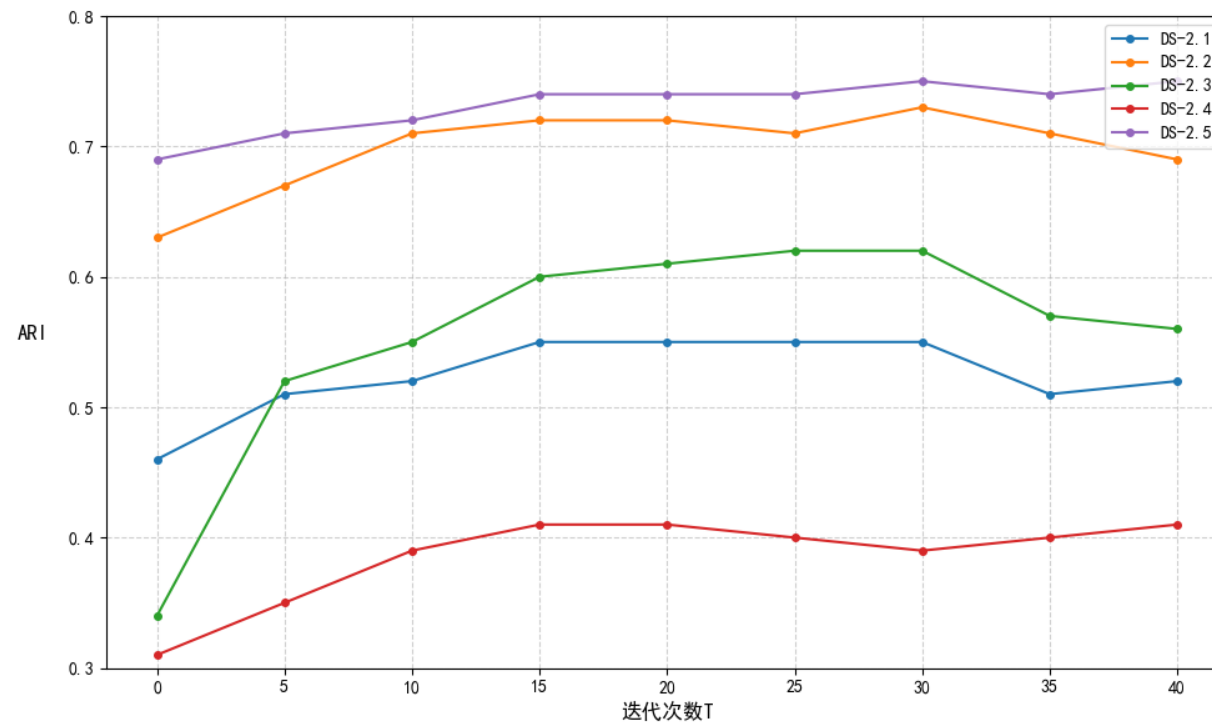
- 参数设置:
 - 采样收敛阈值 ε :





4. 实验环境与设置

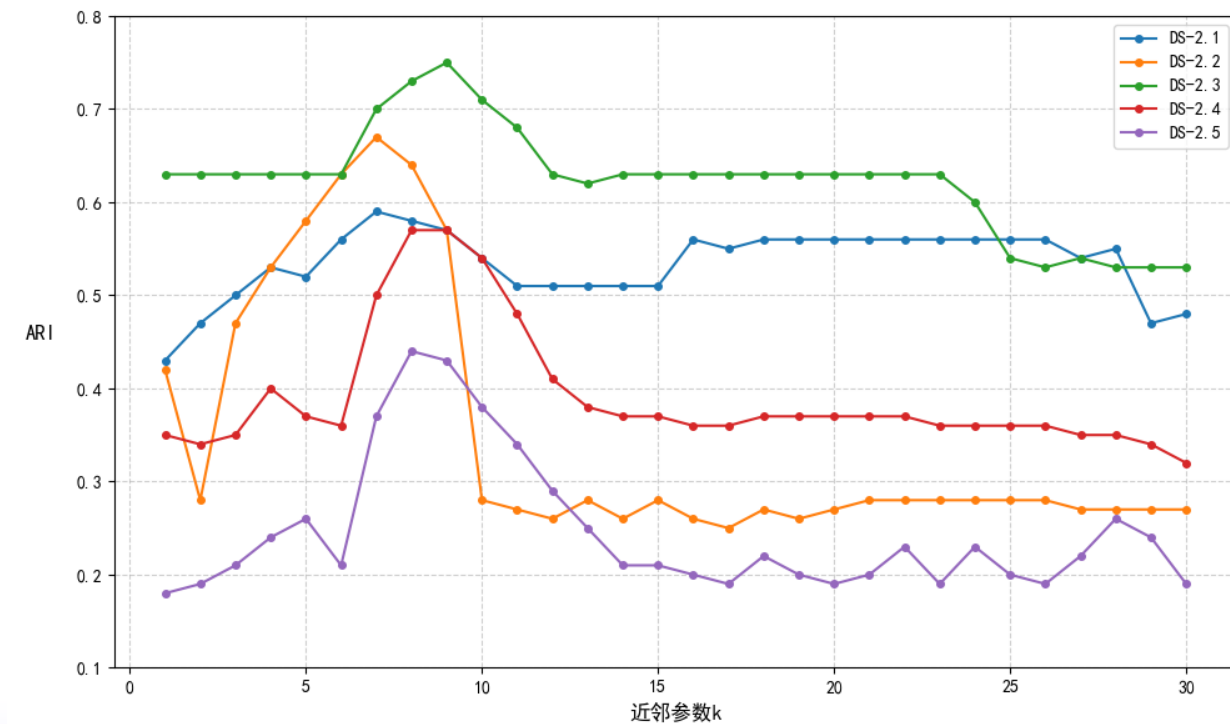
- 参数设置:
 - 迭代次数 T :





4. 实验环境与设置

- 参数设置:
 - 近邻参数 k :





目录

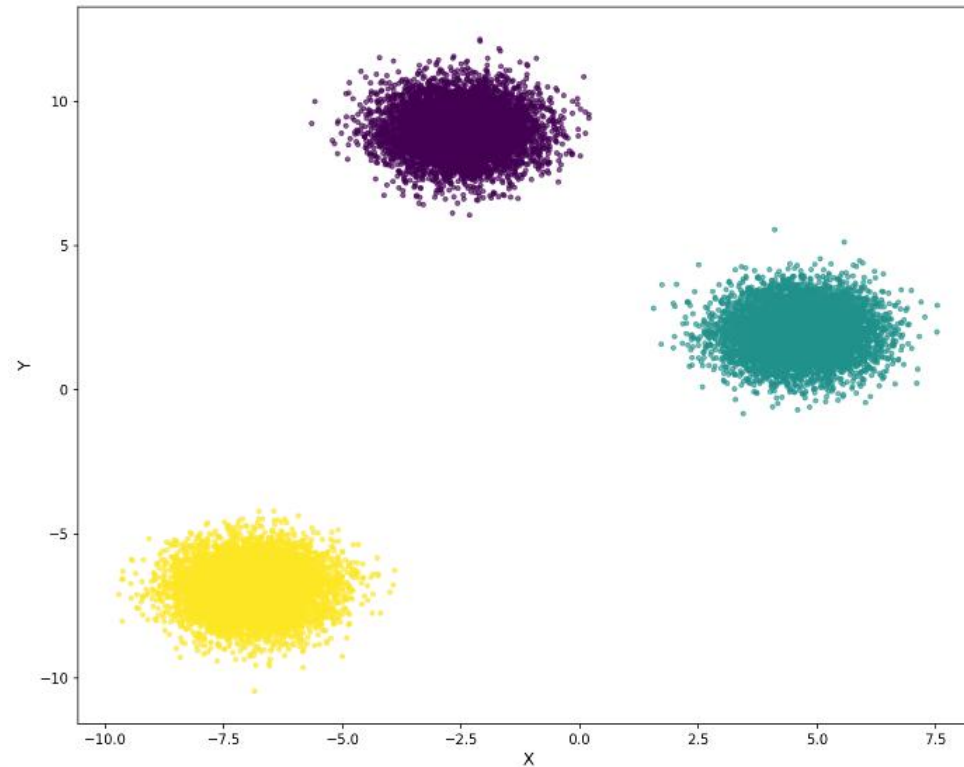
1. 研究背景与意义
2. FASCA算法设计思路
3. 算法总结与分析
4. 实验环境与设置
- 5. 实验结果分析**
- 6. 总结与展望**



5. 实验结果分析

定性分析:

- 球形分布的数据集:

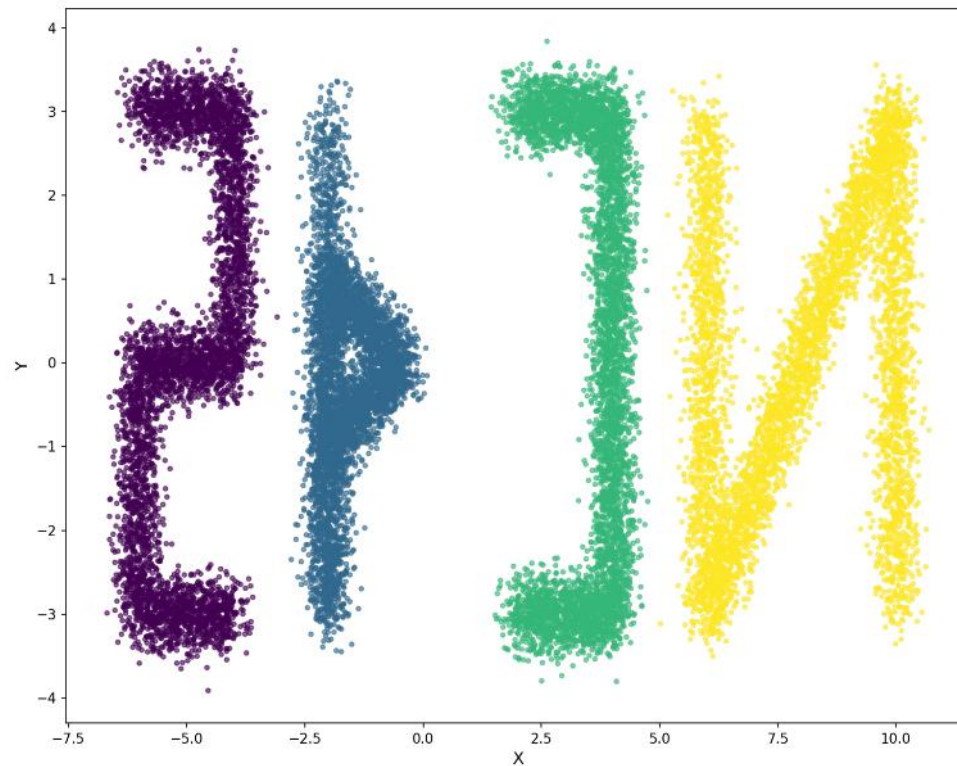




5. 实验结果分析

定性分析:

- 不规则形状分布的数据集:

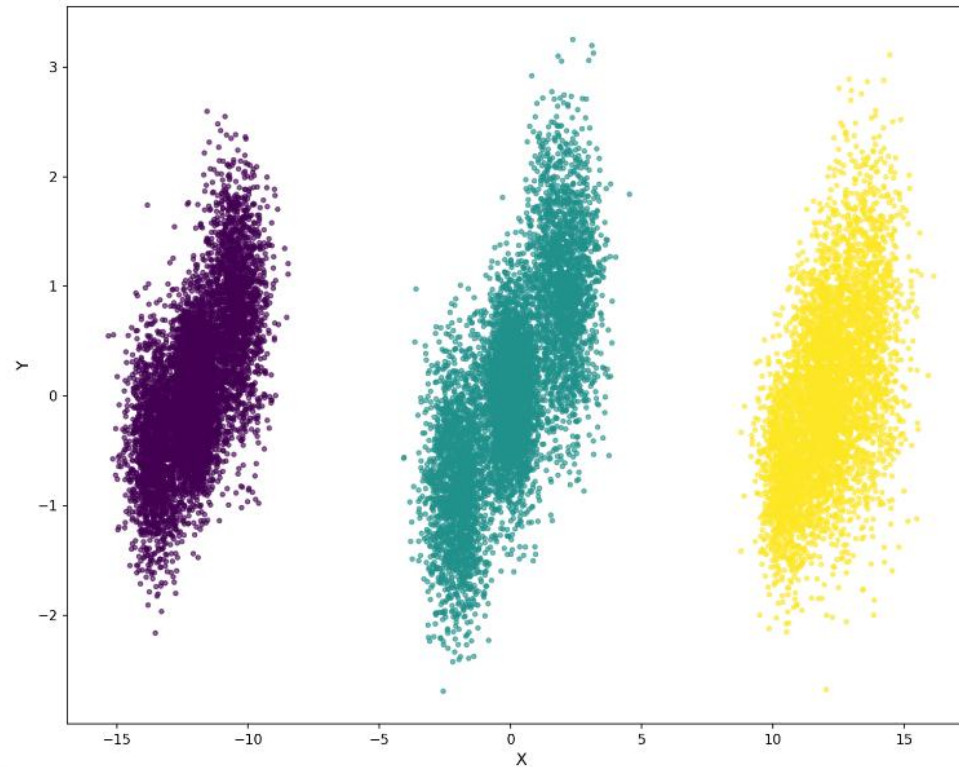




5. 实验结果分析

定性分析:

- 簇内密度不均匀的数据集:

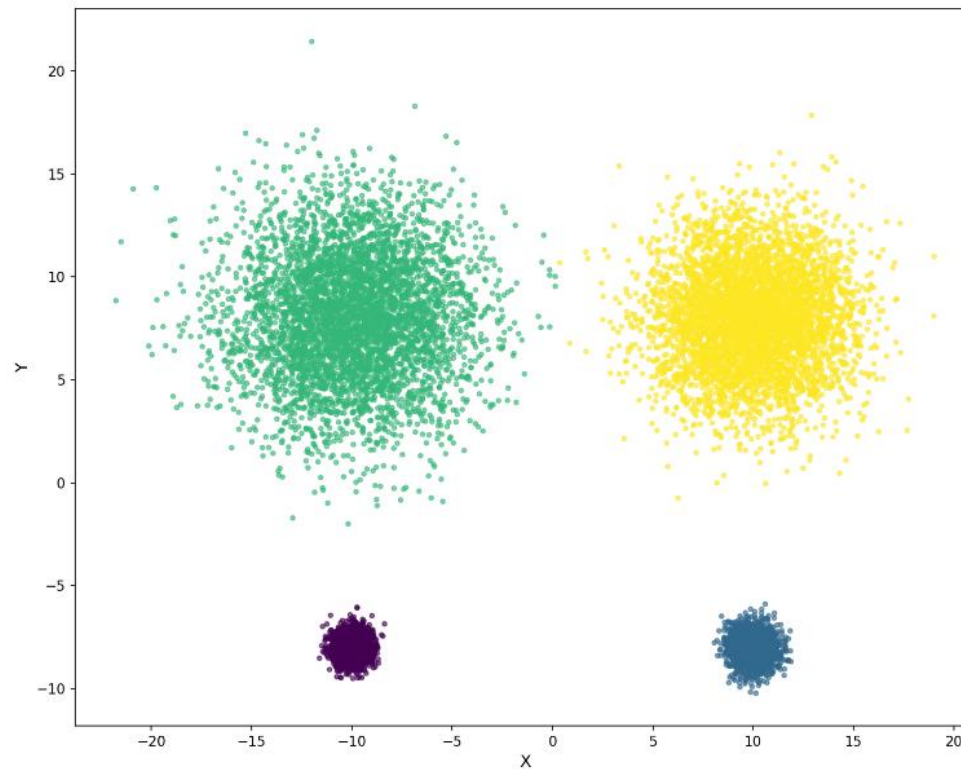




5. 实验结果分析

定性分析:

- 簇间密度不一致的数据集:

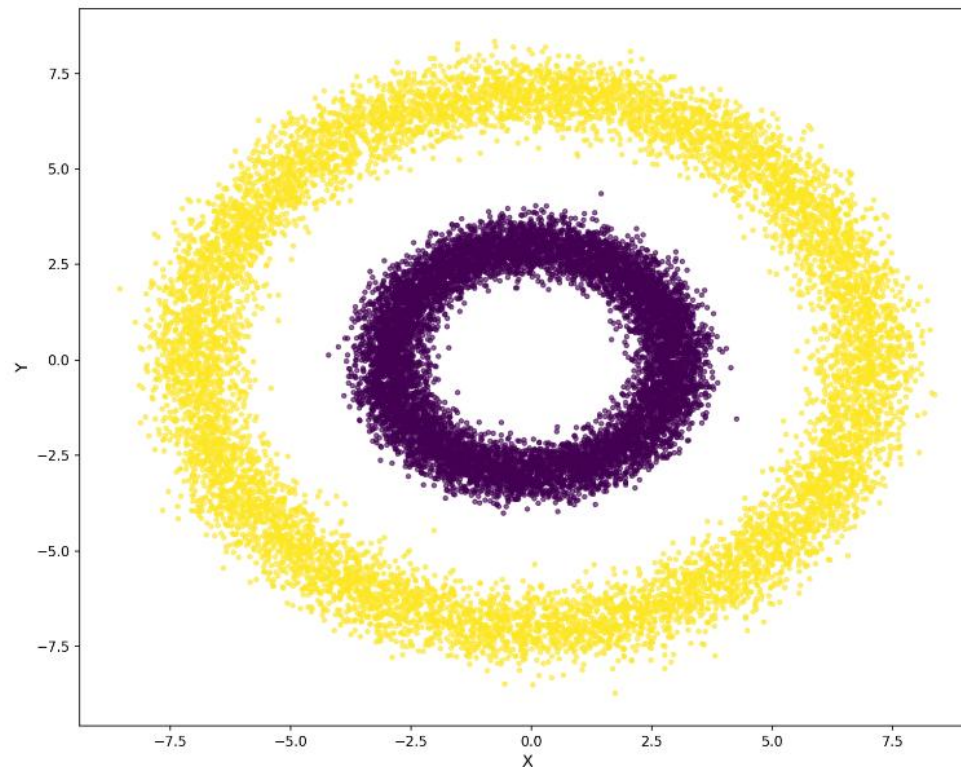




5. 实验结果分析

定性分析:

- 同心环形分布的数据集:





5. 实验结果分析

定量分析:

- 算法ARI对比:

数据集	K-means	DBSCAN	CURE	HK-means	AP	Spectral	FASCA
DS-2.1	0.290	0.201	0.432	0.417	0.120	0.406	0.572
DS-2.2	0.459	0.424	0.515	0.498	0.242	0.467	0.763
DS-2.3	0.187	0.134	0.245	0.327	0.103	0.218	0.564
DS-2.4	0.132	0.189	0.201	0.311	0.081	0.192	0.321
DS-2.5	0.781	0.852	0.989	0.972	0.812	0.874	0.956
DS-2.6	0.137	0.112	0.243	0.170	0.065	0.154	0.342
DS-2.7	0.142	0.137	0.285	0.188	0.113	0.170	0.412
DS-2.8	0.117	0.124	0.198	0.154	0.051	0.137	0.293



5. 实验结果分析

定量分析:

• 算法运行效率对比:

数据集	K-means	DBSCAN	CURE	HK-means	AP	Spectral	FASCA
DS-2.1	0.008	0.101	0.145	0.132	0.228	0.250	0.121
DS-2.2	12.958	20.132	73.742	36.758	135.123	167.572	10.425
DS-2.3	9.154	17.295	30.468	26.642	40.145	38.538	9.672
DS-2.4	0.383	1.452	1.860	1.538	4.125	6.512	1.242
DS-2.5	70.750	455.964	677.752	500.230	900.651	992.732	50.648
DS-2.6	9.860	12.728	53.562	16.262	163.865	174.174	8.791
DS-2.7	0.126	0.157	0.172	0.192	0.272	0.293	0.141
DS-2.8	0.251	0.378	0.575	0.425	1.620	1.845	0.231



目录

1. 研究背景与意义
2. FASCA算法设计思路
3. 算法总结与分析
4. 实验环境与设置
5. 实验结果分析
6. 总结与展望



6. 总结与展望

- **结论：**

- FASCA算法通过代表点采样、代表点位置调整策略，在聚类结果准确性和计算效率间取得平衡。

- **优势：**

- **高效：**代表点采样策略大大降低了算法时间复杂度。
- **准确：**最远端采样策略能保持原数据结构；物理模型能够使得同簇间的数据更加聚拢。
- **抗噪：**快速离群点检测算法能够有效去除背景噪声。



6. 总结与展望

- **劣势：**

- **参数敏感：**无法自适应地决定聚类个数、代表点数量等，需要人工调参。
- **无法动态聚类：**目前的算法主要针对静态数据集，一旦数据发生变动就需要重新聚类分析。
- **维度灾难：**算法使用了KD-Tree划分，在高维数据上仍然会降低效率。

- **展望：**

- **自适应参数：**未来可探索自适应参数调整机制。
- **提高效率：**引入并行计算等策略提升处理速度。
- **适应动态数据：**探索如何将FASCA扩展至动态演变的数据场景中。



谢谢观看！